Robust Machine Learning under Distribution Shift

Seungjae Shin
Qualcomm AI Research, Quantization system team

02. 21. 2025.

Speaker Introduction

- Seungjae Shin
  - Bachelor/MS/Ph.D in KAIST Industrial & Systems Engineering
  - Research Interests: Model/Dataset Compression and Robust ML

- Selected publications
  - Unknown Domain Inconsistency Minimization for Domain Generalization (ICLR2024, co-first authored)
  - Loss Curvature Matching for Dataset Selection and Condensation (AISTATS2023, co-first authored)
  - Frequency domain-based Dataset Distillation (NeurIPS2023, co-first authored)
  - From noisy Prediction to True label : noisy prediction calibration via generative model (ICML2022, co-first authored)

Contents

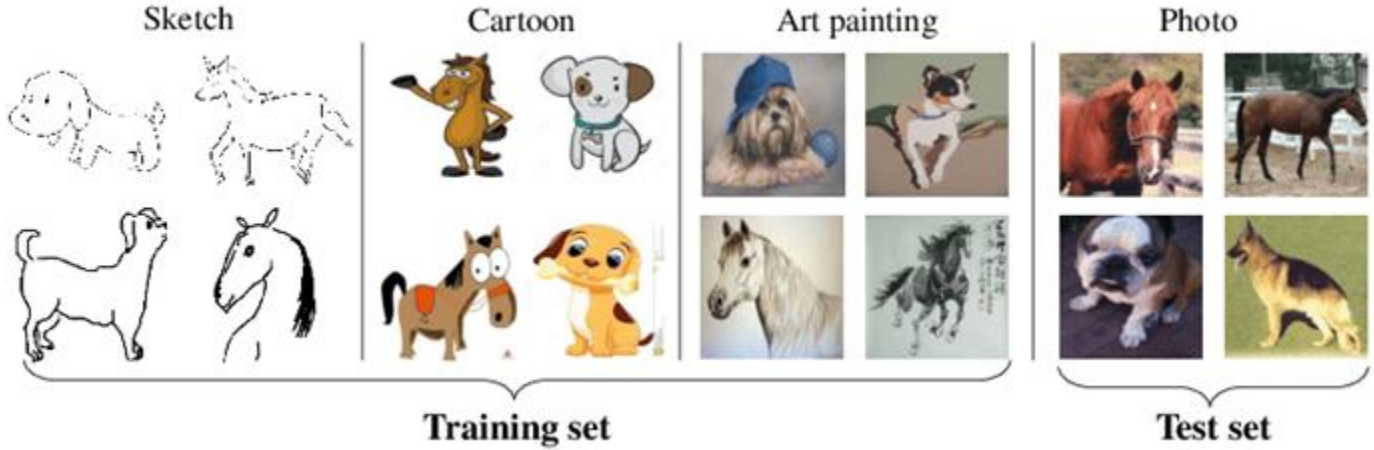
- Introduction to Distribution Shift

- Methods for resolving distribution shift problem
  - Optimization-based methods
  - Weight-averaging methods

Introduction to distribution shift

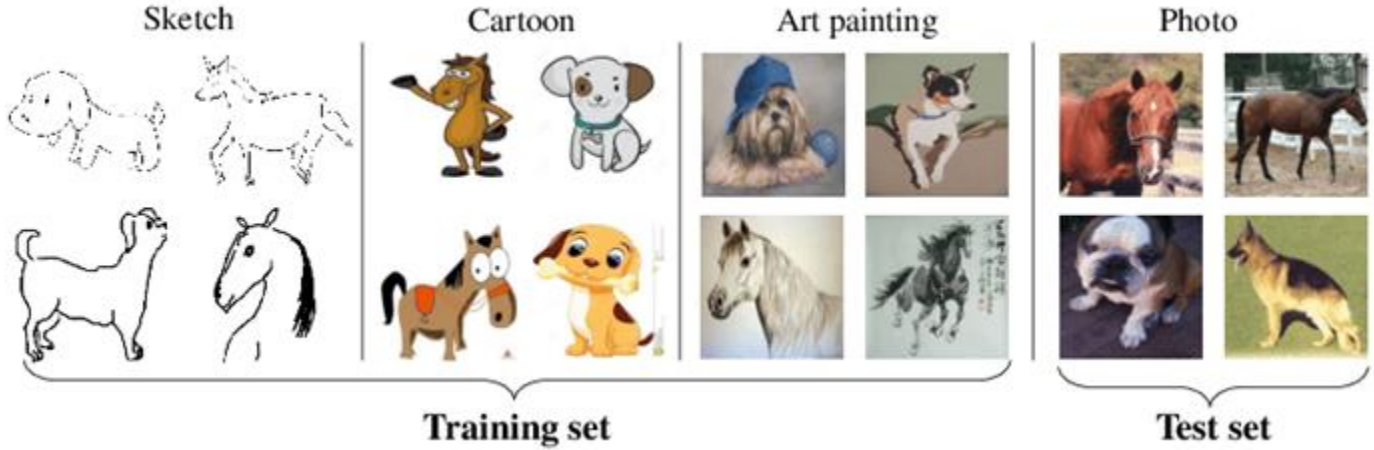
Machine Learning

- Machine learning is widely used for real-world predictions across various domains.
- **Generalization** : Optimizing parameters is crucial for achieving good performance on both the training dataset and new, unseen data.



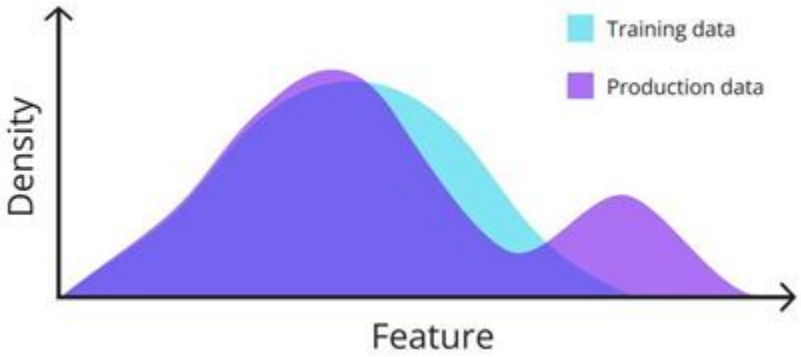
Machine Learning

- Machine learning is widely used for real-world predictions across various domains.
- **Generalization** : Optimizing parameters is crucial for achieving good performance on both the training dataset and new, unseen data.



Distribution Shift

Situation where Distribution used during training (P_{train}) differs from the distribution encountered during testing or deployment (P_{test}).



Distribution Shift

So, Train Dataset and Test Dataset is different ($D_{train} \neq D_{test}$)

The Key Issue is …

The model is optimized under D_{train}

$$\mathbb{E}_{(x,y)\sim D_{train}} \left[\ell\left(f_\theta\left(x\right), y\right)\right]$$

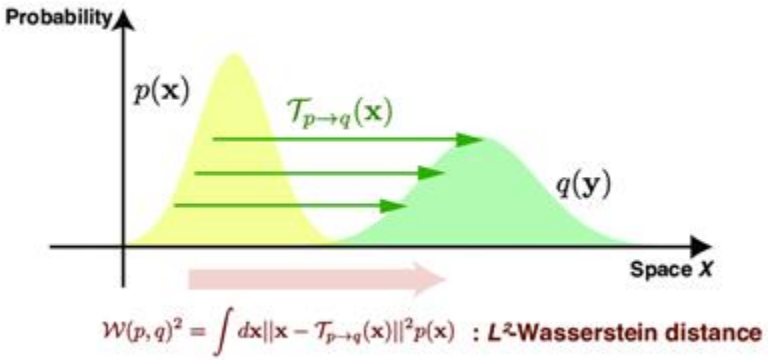
But real-world performance is evaluated under D_{test}

$$\mathbb{E}_{(x,y)\sim D_{test}} \left[\ell\left(f_\theta\left(x\right), y\right)\right]$$

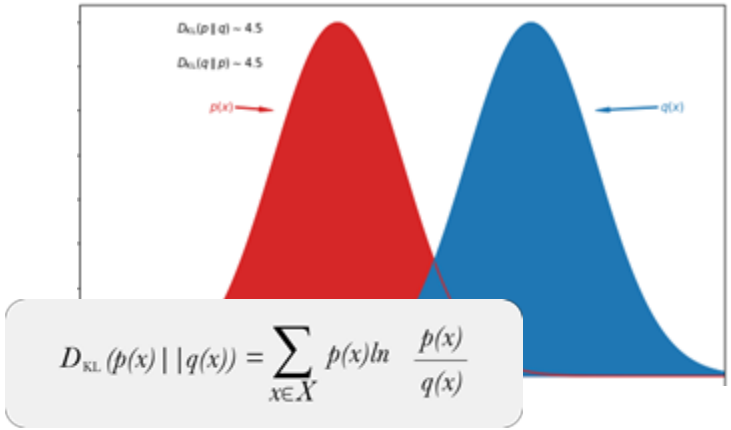
In real-world, data can change over time or come from different domains, leading to potential **drops in performance and stability**.

Quantifying Distribution Shift

- There are methods to quantify the difference between two distributions.
- In practice, we often do not know the full underlying distributions and rely on sample-based estimates or proxy metrics (e.g., FID, MMD).



Wasserstein distance



KL divergence

Quantifying Distribution Shift

- If $P_{train} \neq P_{test}$, there is often a significant gap between R_{train} and R_{test}
- R (Risk) : Expected loss (or error) of a model when making predictions

$$R_{train}(\theta) = \mathbb{E}_{(x,y)\sim P_{train}}\left[\ell\left(f_\theta\left(x\right), y\right)\right]$$

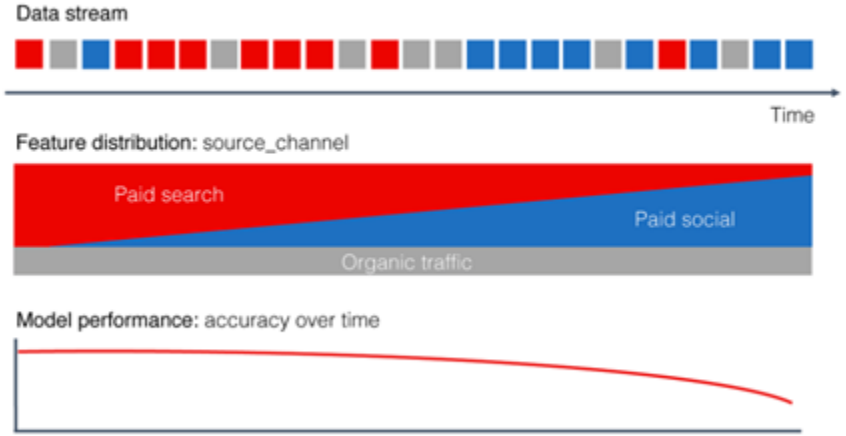
$$R_{test}(\theta) = \mathbb{E}_{(x,y)\sim P_{test}}\left[\ell\left(f_\theta\left(x\right), y\right)\right]$$

Types of Distribution Shift

- Covariate Shift : $P_{train}(x) \neq P_{test}(x)$ while $P(y \mid x)$ remains the same.
  - Example: Images might differ in lighting, background, or resolution (change in input distribution), but the labeling rule is unchanged.

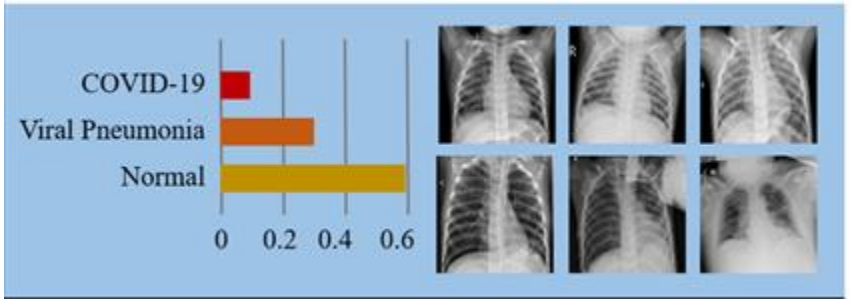
Types of Distribution Shift

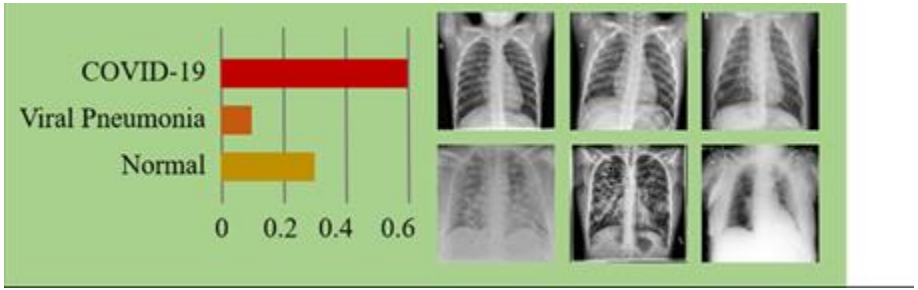
- Covariate Shift : $P_{train}(x) \neq P_{test}(x)$ while $P(y \mid x)$ remains the same.
  - Example: The distribution of categorical variables (e.g., marketing channel types) and the distribution characteristics of continuous variables (e.g., mean and variance of age) change over time, leading to a gradual decline in model performance.



Types of Distribution Shift

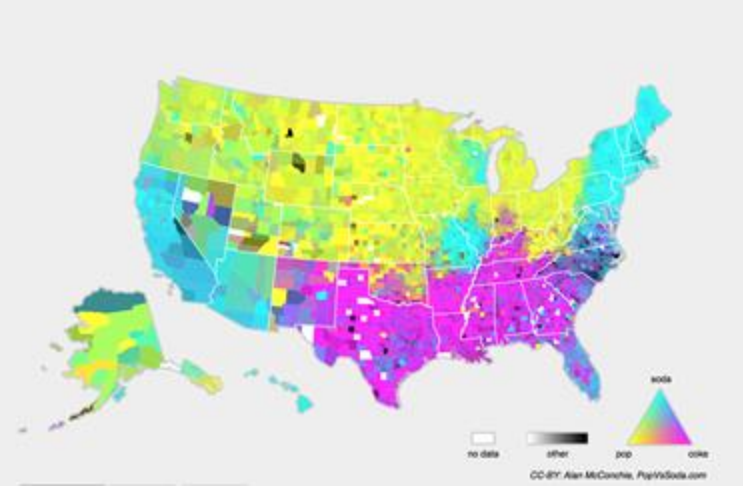
- Label Shift : $P_{train}(y) \neq P_{test}(y)$ while $P(x \mid y)$ remains the same.
  - A class distribution changes over time, so one label becomes more or less frequent in the test set.
  - Example : Train on data with few covid-19 patients. But Test on data during covid-19 season where $P_{train}(y = \text{covid-19}) < P_{test}(y = \text{covid-19})$ while flu symptoms $P(\text{symptoms} \mid \text{covid-19})$ are still the same

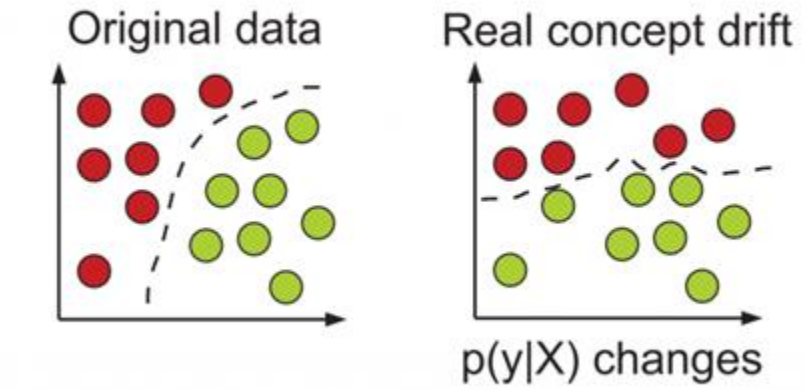




Types of Distribution Shift

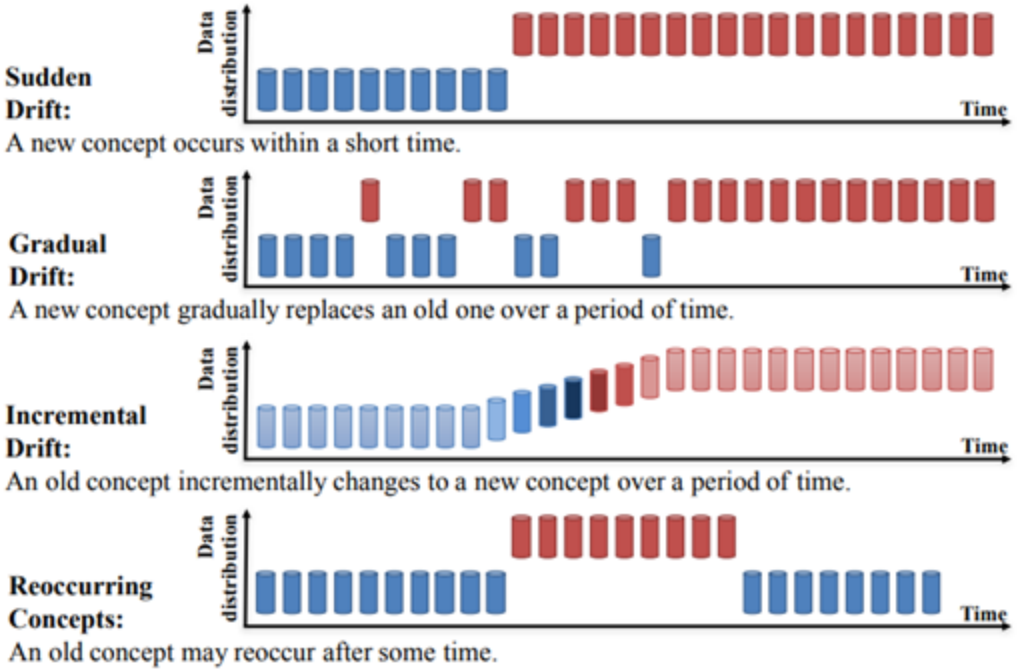
- Concept Shift : $P(y \mid x)$ itself changes (the labeling rule is different)
  - Example : In machine translation, the dataset may contain the same words across different regions, but the terms used to refer to the same object can vary.
  - In the U.S., "soda", "pop", and "soft drink" all refer to the same beverage, but different regions use different terms.





Types of Distribution Shift

- Concept Shift occurs from various reasons, and impact model performances.



Types of Distribution Shift

- Beyond these, there are many types of Distribution Shift
  - **Subpopulation Shift:** Occurs when a specific subgroup (e.g., a particular ethnicity, region, etc.) has insufficient data or a different distribution compared to the overall dataset.
  - **Domain Shift:** Happens when the source domain (training data) and the target domain (deployment environment) are different.
    - Example : CIFAR and applying it to ImageNet, where the image domains differ.

Why is Distribution Shift important?

- Over time, deployed models may degrade in performance if the data distribution evolves (“model drift”).
- When expanding to new domains (e.g., applying a model trained in one country to data from another), performance can drop significantly.
- There are several methods to solve Distribution Shift Problem.
- **From countless methods, I will focus on two methods.**
  - Optimization-based methods
  - Weight-averaging techinques

Example tasks based on distribution shift

1. **Domain Adaptation (Image Classification)**
    - Train on one domain (e.g., clear daytime photos) and test on a different domain (e.g., nighttime or rainy scenes).
2. **Time-Series Forecasting with Concept Drift**
    - Predict future data based on historical patterns that may no longer hold true (e.g., due to economic or behavioral shifts).
3. **Label Shift (Class Imbalance Changes)**
    - Class frequencies change dramatically from training to deployment (e.g., a rare disease becoming more prevalent).
4. **Cross-Lingual NLP (Transfer Across Languages or Domains)**
    - Trained in one language or domain (e.g., English product reviews), tested in another (e.g., Spanish social media posts).

Methods for resolving distribution shift

#1 Optimization-based methods

How to update model?

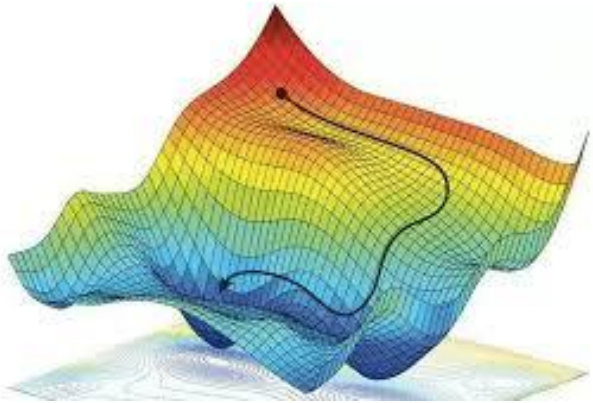
- **Model parameters update via gradient descent**

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{1}{|B|} \sum_{(x_i, y_i) \in B} \nabla_\theta \ell\left(f_{\theta_t}(x_i),\, y_i\right)$$

  - Given our model parameter θ, we update model via given gradient.

Gradient & Hessian Matrix

- **Gradient (First-Order Derivative)**
  - Gradient indicates the **direction of the steepest ascent** (or descent, if using negative gradient)
  - In machine learning and optimization, the gradient is **crucial for updating model parameters,** typically in gradient-based optimization methods like Gradient Descent.

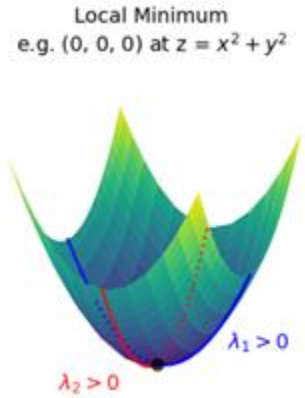


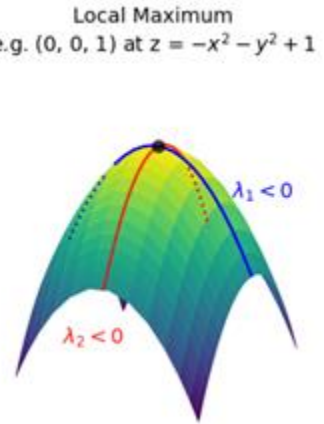
$$\nabla f = \begin{pmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{pmatrix}$$

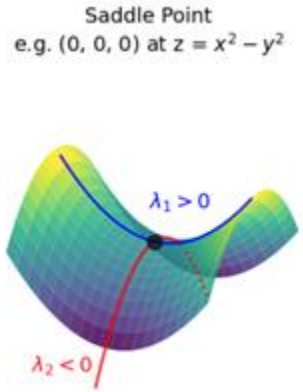
Gradient & Hessian Matrix

- **Hessian Matrix (Second-Order Derivative)**
  - Provides information about the **curvature of the function.**
  - **Eigenvector** of the Hessian Matrix represents the **principal direction** in which the function's curvature changes
  - **The eigenvalues (λ)** indicate **how the function curves** in different directions:

$$\nabla^2 f = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{pmatrix}$$







KAIST

Eigen-decomposition of the Hessian

- **Interpreting the Eigenvalues as “sharpness”**

  Because $H(\mathbf{x}_0)$ is a real symmetric matrix, it can be diagonalized via an orthogonal (orthonormal) matrix:

$$H(\mathbf{x}_0) = Q\,\Lambda\,Q^\top, \quad Q^\top Q = I,$$

  where $\Lambda = diag(\lambda 1, \ldots, \lambda n)$ contains the **eigenvalues** λ_i,

  and the columns of Q are the corresponding **eigenvectors** q_i.

  If we define
  $\Delta\mathbf{y} = Q^\top \Delta\mathbf{x}$, then the quadratic form becomes

$$\Delta\mathbf{x}^\top H(\mathbf{x}_0)\,\Delta\mathbf{x} = \Delta\mathbf{y}^\top \Lambda\,\Delta\mathbf{y} = \sum_{i=1}^{n} \lambda_i\,(\Delta y_i)^2.$$

Eigen-decomposition of the Hessian

- **Interpreting the Eigenvalues as “sharpness”**

  - **Eigenvalues $\lambda_i > 0$**: The function is *locally convex* in the direction of the corresponding eigenvector. A larger positive implies a *sharper* (steeper) curvature in that direction.
  - **Eigenvalues $\lambda_i < 0$**: The function is *locally concave* in that direction, and a more negative indicates a sharper downward curvature.
  - **Eigenvalue $\lambda_i = 0$**: No second-order curvature in that direction.

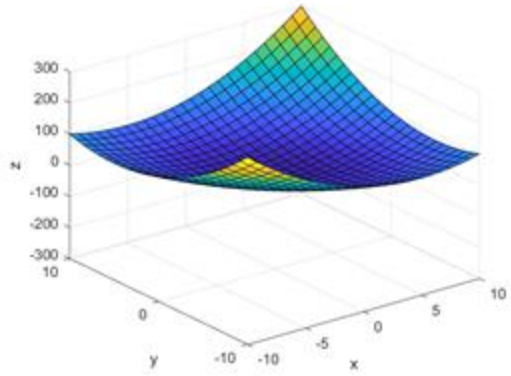
  Essentially, the magnitude $|\lambda_i|$ indicates how quickly the function curves (changes) in the eigenvector direction q_i. Thus, **the eigenvalues of the Hessian directly measure the “sharpness” of curvature** in their respective principal directions.

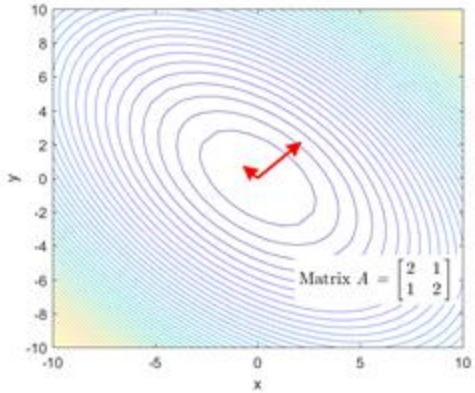
Gradient & Hessian Matrix

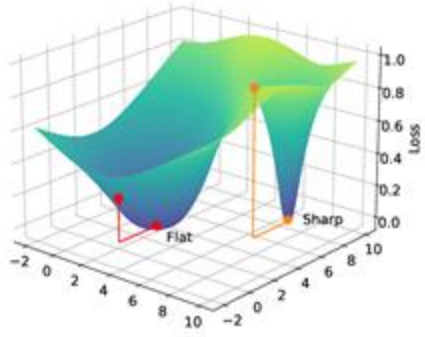
- **Sharpness :** Steepness of the curvature around the minimum of the loss function.
  - **Sharp minimum:** Curvature is high, and the gradient changes rapidly.
  - **Flat minimum:** Curvature is low, and the changes in the gradient are more gradual.

- **Eigenvalue** quantifies the **sharpness of that curvature** in that direction.

$$Sharpness = \lambda_{\max}\left(\nabla^2 f\right)$$







KAIST

Why Gradient & Hessian Matter?

- **Risk minimization under shifting distributions**
  - Let $R(\theta) = \mathbb{E}_{(x,y)\sim P}[\ell(f_\theta(x), y)]$ be the expected loss under a given distribution.
  - If $P_{train}(x) \neq P_{test}(x)$, the point that minimizes train loss may not minimize test loss.

- **Sensitivity via Second-order expansion (based on taylor expansion)**
  - Let R(\theta) = \mathbb{E}_{(x,, be the expected loss under a given distribution.

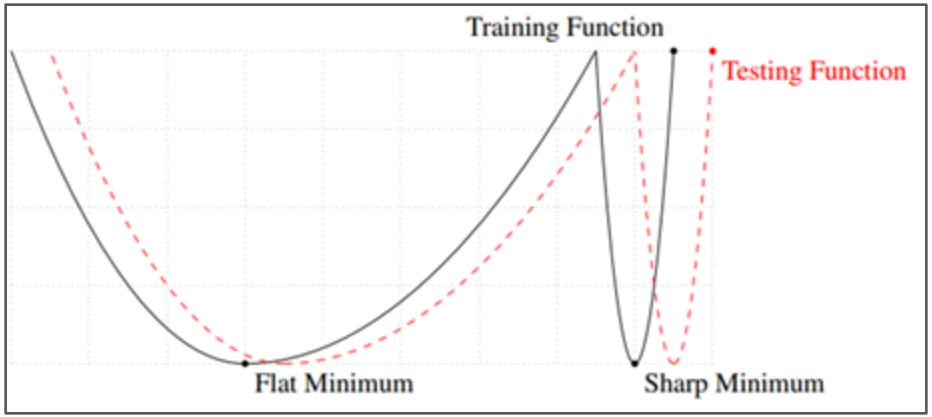
$$R(\theta + \Delta) \approx R(\theta) + \nabla R(\theta)^\top \Delta + \tfrac{1}{2}\, \Delta^\top \mathbf{H}(\theta)\, \Delta,$$

- $\nabla R(\theta)$ = **Gradient** at θ
- $\mathbf{H}(\theta)$ = **Hessian** matrix

- **Robustness & Curvature**
  - Finding a “flatter” region (lower eigenvalues) can reduce sensitivity to shifts because parameter perturbations (or small distribution changes) won’t sharply increase the loss.

Sharpness : Parameter-region based information

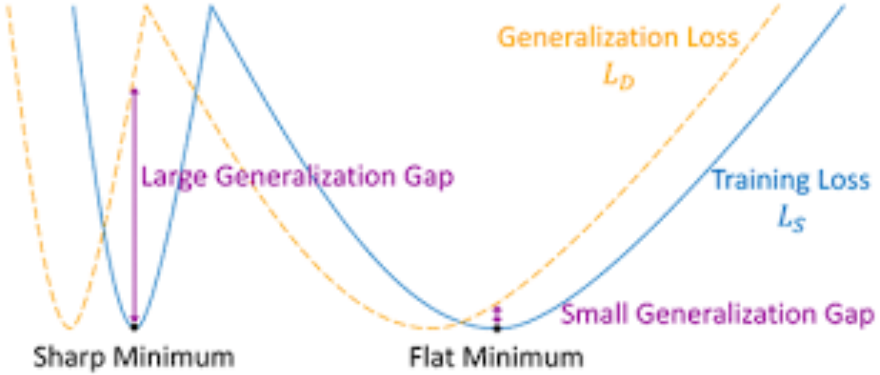
- **Parameter-region based information**
  - Information obtained from a **locally perturbed region** of a current parameter point.
  - The **sharpness** is defined with respect to the loss($L(D;\theta)$) change based on the parameter perturbation in the loss landscape, which corresponds to local region of θ



$$Sharpness = L(D;\theta+\epsilon) - L(D;\theta)$$

Sharpness : Parameter-region based information

- **Flat minima** leads to **better generalization**
- **Conceptual sketch of flat and sharp minima :**
  ⇒ The loss error remains approximately constant in the parameter-space near the flat minima.



Then, what is worst-case sharpness?

$$\left[\max_{\|\epsilon\|\leq\rho} L(D_T;\theta+\epsilon) - L(D_T;\theta)\right]$$

where $\rho \geq 0$ is the perturbation size.

Sharpness-Aware Minimization parameter region-based optimization

- High correlation of Sharpness and generalization leads to **Sharpness-Aware Minimization (SAM)**
  - Minimizing maximally perturbed loss leads to minimizing **both 1) loss sharpness and 2) original loss**

Sharpness-Aware Minimization (SAM)

$$\min_{\theta} \max_{\|\epsilon\|\leq\rho} L(D_T;\theta+\epsilon)+\lambda\|\theta\|_2^2$$

where $\rho \geq 0$ is a hyperparameter that controls the perturbation size.

- The first term of objective is decomposed into **sharpness term** and loss term

$$\max_{\|\epsilon\|\leq\rho} L(D_T;\theta+\epsilon) = \underbrace{\left[\max_{\|\epsilon\|\leq\rho} L(D_T;\theta+\epsilon) - L(D_T;\theta)\right]}_{\text{Loss Sharpness}} + \underbrace{L(D_T;\theta)}_{\text{Loss}}$$

Sharpness-Aware Minimization parameter region-based optimization

- Finally, **Generalization loss(Population risk)**, $L(\mathbb{D};\theta)$, is upper bounded by the **maximally perturbed loss** of training dataset

Theorem. [4] (Generalization bound w.r.t. neighborhood-wise training loss)

For any $\rho > 0$, with high probability over training dataset D_T generated from population dataset $\mathbb{D}_p$,

$$L(\mathbb{D};\theta) \leq \max_{\|\epsilon\|\leq\rho} L(D_T;\theta+\epsilon) + h(\|\theta\|_2^2/\rho^2),$$

where $h: \mathbb{R}_+ \to \mathbb{R}_+$ is a strictly increasing function.

$\mathbb{D}$: population distribution
D_T : training dataset
ℓ : loss function
θ: model parameter

Implementation of SAM

- Implementation includes min-max structure.

$$\min_{\theta} L^{SAM}(\theta) + \lambda\|\theta\|_2^2, \text{ where } L^{SAM}(\theta) \triangleq \max_{\|\epsilon\|\leq\rho} L(\theta+\epsilon)$$

- Solve Inner maximization problem via a **first-order Taylor expansion**.
  - $\epsilon^* = \underset{\|\epsilon\|_2\leq\rho}{\operatorname{argmax}} L(\theta+\epsilon) \approx \underset{\|\epsilon\|_2\leq\rho}{\operatorname{argmax}}\{L(\theta) + \epsilon^T\nabla_\theta L(\theta)\} = \underset{\|\epsilon\|_2\leq\rho}{\operatorname{argmax}} \epsilon^T\nabla_\theta L(\theta)$
  - $\hat{\epsilon}(\theta) = \rho \cdot sign(\nabla_\theta L(\theta)) \cdot \frac{|\nabla_\theta L(\theta)|^{q-1}}{\left(\|\nabla_\theta L(\theta)\|_q^q\right)^{1/p}}$, where $\frac{1}{p}+\frac{1}{q}=1$
- Solve Outer minimization problem : **Get gradient** as below.
  - $\nabla_\theta L^{SAM}(\theta) \approx \nabla_\theta L\left(\theta+\hat{\epsilon}(\theta)\right)$

$$= \frac{d(\theta+\hat{\epsilon}(\theta))}{d\theta}\nabla_\theta L(\theta)|_{\theta+\hat{\epsilon}(\theta)}$$

$$= \nabla_\theta L(\theta)|_{\theta+\hat{\epsilon}(\theta)} + \frac{d\hat{\epsilon}(\theta)}{d\theta}\nabla_\theta L(\theta)|_{\theta+\hat{\epsilon}(\theta)}$$

$$\approx \nabla_\theta L(\theta)|_{\theta+\hat{\epsilon}(\theta)}$$

Implementation of SAM

- Computational costs of SAM
  - SAM computes **two gradients** per update, one at the current parameter values, and one at the perturbed parameters.

Input: Training set $\mathcal{S} \triangleq \cup_{i=1}^{n}\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}$, Loss function $l : \mathcal{W} \times \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}_+$, Batch size b, Step size $\eta > 0$, Neighborhood size $\rho > 0$.
Output: Model trained with SAM
Initialize weights $\boldsymbol{w}_0$, $t = 0$;
while *not converged* **do**
  Sample batch $\mathcal{B} = \{(\boldsymbol{x}_1, \boldsymbol{y}_1), ...(\boldsymbol{x}_b, \boldsymbol{y}_b)\}$;
  Compute gradient $\nabla_{\boldsymbol{w}} L_{\mathcal{B}}(\boldsymbol{w})$ of the batch's training loss;
  Compute $\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})$ per equation 2;
  Compute gradient approximation for the SAM objective (equation 3): $\boldsymbol{g} = \nabla_{w} L_{\mathcal{B}}(\boldsymbol{w})|_{\boldsymbol{w}+\hat{\boldsymbol{\epsilon}}(\boldsymbol{w})}$;
  Update weights: $\boldsymbol{w}_{t+1} = \boldsymbol{w}_t - \eta \boldsymbol{g}$;
  $t = t + 1$;
end
return $\boldsymbol{w}_t$

Algorithm 1: SAM algorithm

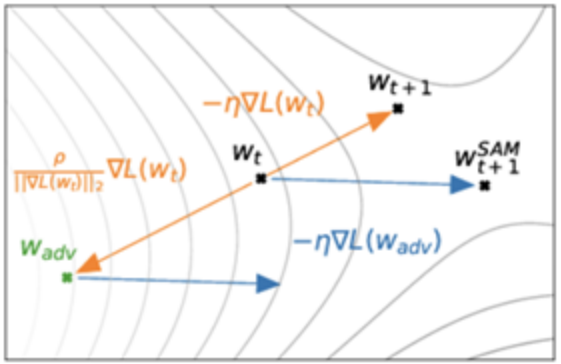


Figure 2: Schematic of the SAM parameter update.

KAIST

Correlation of Loss Sharpness with Generalization

- Loss sharpness relatively shows higher correlation with generalization (test accuracies) than other metrics or optimization methods.

		batch size	dropout	learning rate	depth	optimizer	weight decay	width	overall τ	Ψ
Corr	Frob distance 40	-0.317	-0.833	-0.718	0.526	-0.214	-0.669	-0.166	-0.263	-0.341
	Spectral orig 26	-0.262	-0.762	-0.665	-0.908	-0.131	-0.073	-0.240	-0.537	-0.434
	Parameter norm 42	0.236	-0.516	0.174	0.330	**0.187**	0.124	-0.170	0.073	0.052
	Path norm 44	0.252	**0.270**	0.049	**0.934**	0.153	0.338	**0.178**	**0.373**	**0.311**
	Fisher-Rao 45	**0.396**	0.147	**0.240**	-0.553	0.120	**0.551**	0.177	0.078	0.154
	oracle 0.02	0.380	0.657	0.536	0.717	0.374	0.388	0.360	0.714	0.487
Corr	sharpness-orig 52	0.542	-0.359	0.716	0.816	0.297	0.591	0.185	0.400	0.398
	pacbayes-orig 49	0.526	-0.076	0.705	0.546	**0.341**	0.564	-0.086	0.293	0.360
	$1/\alpha'$ sharpness mag 62	**0.570**	**0.148**	**0.762**	0.824	0.297	**0.741**	**0.269**	**0.484**	**0.516**
	$1/\sigma'$ pacbayes mag 61	0.490	-0.215	0.505	**0.896**	0.186	0.147	0.195	0.365	0.315
	oracle 0.02	0.380	0.657	0.536	0.717	0.374	0.388	0.360	0.714	0.487
Corr	step to 0.1 63	-0.664	-0.861	-0.255	**0.440**	-0.030	-0.628	0.043	-0.264	-0.279
	step 0.1 to 0.01 64	-0.151	-0.069	-0.014	0.114	0.072	-0.046	-0.021	-0.088	-0.016
	grad noise 1 epoch 65	0.071	**0.378**	0.376	-0.517	0.121	0.221	0.037	0.070	0.098
	grad noise final 66	**0.452**	0.119	**0.427**	0.141	**0.245**	**0.432**	**0.230**	**0.311**	**0.292**
	oracle 0.02	0.380	0.657	0.536	0.717	0.374	0.388	0.360	0.714	0.487

Correlation results of generalization and diverse metrics based on different conbinations of learning hyper-parameters (This paper utilizes test accuracies as a generalization metric)

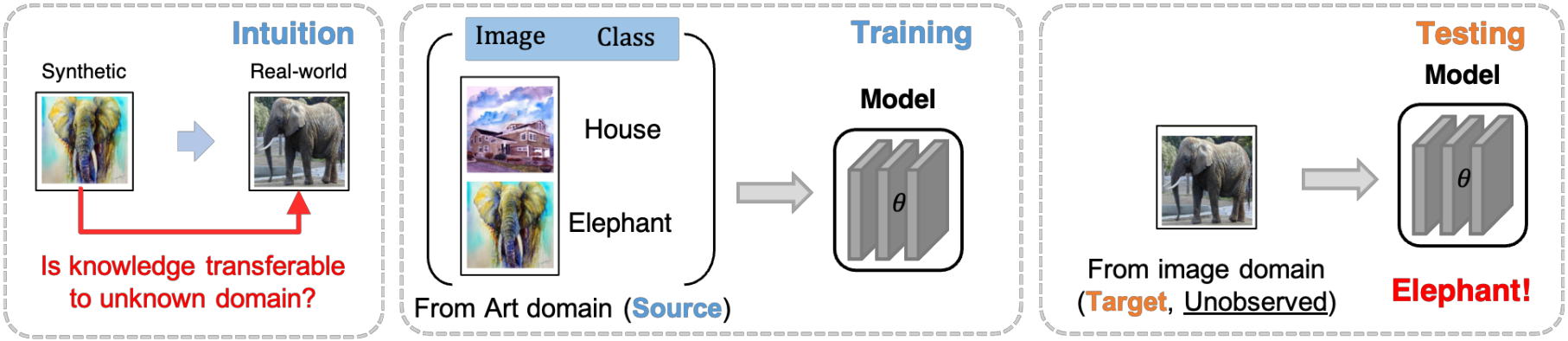
γ denotes the correlation coefficient between generalization and specific metric

Experimental results on cifar-10/cifar-100

		CIFAR-10		CIFAR-100	
Model	Augmentation	SAM	SGD	SAM	SGD
WRN-28-10 (200 epochs)	Basic	**2.7**$_{\pm 0.1}$	3.5$_{\pm 0.1}$	**16.5**$_{\pm 0.2}$	18.8$_{\pm 0.2}$
WRN-28-10 (200 epochs)	Cutout	**2.3**$_{\pm 0.1}$	2.6$_{\pm 0.1}$	**14.9**$_{\pm 0.2}$	16.9$_{\pm 0.1}$
WRN-28-10 (200 epochs)	AA	**2.1**$_{\pm <0.1}$	2.3$_{\pm 0.1}$	**13.6**$_{\pm 0.2}$	15.8$_{\pm 0.2}$
WRN-28-10 (1800 epochs)	Basic	**2.4**$_{\pm 0.1}$	3.5$_{\pm 0.1}$	**16.3**$_{\pm 0.2}$	19.1$_{\pm 0.1}$
WRN-28-10 (1800 epochs)	Cutout	**2.1**$_{\pm 0.1}$	2.7$_{\pm 0.1}$	**14.0**$_{\pm 0.1}$	17.4$_{\pm 0.1}$
WRN-28-10 (1800 epochs)	AA	**1.6**$_{\pm 0.1}$	2.2$_{\pm <0.1}$	**12.8**$_{\pm 0.2}$	16.1$_{\pm 0.2}$
Shake-Shake (26 2x96d)	Basic	**2.3**$_{\pm <0.1}$	2.7$_{\pm 0.1}$	**15.1**$_{\pm 0.1}$	17.0$_{\pm 0.1}$
Shake-Shake (26 2x96d)	Cutout	**2.0**$_{\pm <0.1}$	2.3$_{\pm 0.1}$	**14.2**$_{\pm 0.2}$	15.7$_{\pm 0.2}$
Shake-Shake (26 2x96d)	AA	**1.6**$_{\pm <0.1}$	1.9$_{\pm 0.1}$	**12.8**$_{\pm 0.1}$	14.1$_{\pm 0.2}$
PyramidNet	Basic	**2.7**$_{\pm 0.1}$	4.0$_{\pm 0.1}$	**14.6**$_{\pm 0.4}$	19.7$_{\pm 0.3}$
PyramidNet	Cutout	**1.9**$_{\pm 0.1}$	2.5$_{\pm 0.1}$	**12.6**$_{\pm 0.2}$	16.4$_{\pm 0.1}$
PyramidNet	AA	**1.6**$_{\pm 0.1}$	1.9$_{\pm 0.1}$	**11.6**$_{\pm 0.1}$	14.6$_{\pm 0.1}$
PyramidNet+ShakeDrop	Basic	**2.1**$_{\pm 0.1}$	2.5$_{\pm 0.1}$	**13.3**$_{\pm 0.2}$	14.5$_{\pm 0.1}$
PyramidNet+ShakeDrop	Cutout	**1.6**$_{\pm <0.1}$	1.9$_{\pm 0.1}$	**11.3**$_{\pm 0.1}$	11.8$_{\pm 0.2}$
PyramidNet+ShakeDrop	AA	**1.4**$_{\pm <0.1}$	1.6$_{\pm <0.1}$	**10.3**$_{\pm 0.1}$	10.6$_{\pm 0.1}$

Deployed task : domain generalization

- Task setting of Domain generalization
  - Utilize a knowledge of training distribution, called **source domain**.
  - To achieve the the predictive ability for the unknown test distribution, called **target domain**.



Deployed task : domain generalization

- Experimental results
  - Even only with optimization, SAM-based approaches show good performances.

Algorithm	PACS	VLCS	OfficeHome	TerraInc	DomainNet	Avg.
MMD† [31]	84.7±0.5	77.5±0.9	66.3±0.1	42.2±1.6	23.4±9.5	58.8
Mixstyle‡ [53]	85.2±0.3	77.9±0.5	60.4±0.3	44.0±0.7	34.0±0.1	60.3
GroupDRO† [41]	84.4±0.8	76.7±0.6	66.0±0.7	43.2±1.1	33.3±0.2	60.7
IRM† [1]	83.5±0.8	78.5±0.5	64.3±2.2	47.6±0.8	33.9±2.8	61.6
ARM† [51]	85.1±0.4	77.6±0.3	64.8±0.3	45.5±0.3	35.5±0.2	61.7
VREx† [28]	84.9±0.6	78.3±0.2	66.4±0.6	46.4±0.6	33.6±2.9	61.9
CDANN† [32]	82.6±0.9	77.5±0.1	65.8±1.3	45.8±1.6	38.3±0.3	62.0
AND-mask [42]	84.4±0.9	78.1±0.9	65.6±0.4	44.6±0.3	37.2±0.6	62.0
DANN† [16]	83.6±0.4	78.6±0.4	65.9±0.6	46.7±0.5	38.3±0.1	62.6
RSC† [21]	85.2±0.9	77.1±0.5	65.5±0.9	46.6±1.0	38.9±0.5	62.7
MTL† [5]	84.6±0.5	77.2±0.4	66.4±0.5	45.6±1.2	40.6±0.1	62.9
Mixup† [49]	84.6±0.6	77.4±0.6	68.1±0.3	47.9±0.8	39.2±0.1	63.4
MLDG† [29]	84.9±1.0	77.2±0.4	66.8±0.6	47.7±0.9	41.2±0.1	63.6
ERM [46]	85.5±0.2	77.3±0.4	66.5±0.3	46.1±1.8	43.8±0.1	63.9
Fish [44]	85.5±0.3	77.8±0.3	68.6±0.4	45.1±1.3	42.7±0.2	63.9
SagNet† [36]	86.3±0.2	77.8±0.5	68.1±0.1	48.6±1.0	40.3±0.1	64.2
SelfReg [25]	85.6±0.4	77.8±0.9	67.9±0.7	47.0±0.3	42.8±0.0	64.2
CORAL† [45]	86.2±0.3	78.8±0.6	68.7±0.3	47.6±1.0	41.5±0.1	64.5
mDSDI [6]	86.2±0.2	79.0±0.3	69.2±0.4	48.1±1.4	42.8±0.1	65.1
Miro [9] (with CLIP [39])	85.4±0.4	79.0±0.0	**70.5**±0.4	**50.4**±1.1	44.3±0.2	65.9
SAM ‡[15]	85.8±0.2	79.4±0.1	69.6±0.1	43.3±0.7	44.3±0.0	64.5
GSAM [55]	85.9±0.1	79.1±0.2	69.3±0.0	47.0±0.8	44.6±0.2	65.1
SAGM (ours)	**86.6**±0.2	**80.0**±0.3	70.1±0.2	48.8±0.9	**45.0**±0.2	**66.1**

KAIST

Example variants on SAM

- Gradient-norm aware minimization (CVPR2023)
  - **Motivation**
    - Many flatness-based methods (e.g., SAM) use *zeroth-order* approximations (e.g., $\max_{\|\delta\| \leq \rho} L(\theta + \delta)$) to find flatter minima.
    - However, the *first-order* perspective (based on gradient norms) can offer a more direct handle on the curvature (i.e., Hessian eigenvalues).
  - **Key Definition: First-order Flatness**
    - Measures the **maximum gradient norm** within a ball ρ around θ.
    - Strongly correlates with the largest Hessian eigenvalue.

$$R_\rho^{(1)}(\theta) \triangleq \rho \cdot \max_{\theta' \in B(\theta, \rho)} \left\| \nabla \hat{L}(\theta') \right\|.$$

Example variants on SAM

- Gradient-norm aware minimization (CVPR2023)
  - **GAM Objective**
    - α controls regularization strength, combining any base loss $\hat{L}(\theta)$ with **first-order flatness**.
  - **Why It Helps**
    - First-order flatness captures *how fast* the loss can increase around θ, offering a potentially *stronger measure* of sharpness.
    - Minimizing $R_\rho^{(1)}(\theta)$ **implicitly bounds** the maximal Hessian eigenvalue ($\lambda_{\max}$) near θ.

$$L_{\text{overall}}(\theta) \;=\; L_{\text{oracle}}(\theta) \;+\; \alpha\, R_\rho^{(1)}(\theta),$$

$$R_\rho^{(1)}(\theta) \;\triangleq\; \rho \cdot \max_{\theta' \in B(\theta,\,\rho)} \left\| \nabla \hat{L}(\theta') \right\|.$$

Example variants on SAM

- SAM vs GAM
  - **SAM (Sharpness-Aware Minimization)**
    - Optimizes a loss that penalizes the maximum increase in function value in a neighborhood of radius ρ.
    - *Zeroth-order* approach: $\max_{\|\delta\| \leq \rho} L(\theta + \delta)$
  - **GAM (Gradient-norm Aware Minimization)**
    - Convergence analysis leverages a **gradient norm** approximation using Hessian-vector products (less costly than full Hessian).
    - *First-order* approach: directly controls the **maximum gradient norm** in the neighborhood.
    - Tighter link to the largest Hessian eigenvalue: $\lambda_{\max}\left(\nabla^2 L(\theta)\right) = \frac{R_\rho^{(1)}(\theta)}{\rho^2}$ (under certain assumptions).

Example variants on SAM

		CIFAR-10				CIFAR-100			
Model	Aug	SGD	SGD + GAM	SAM	SAM + GAM	SGD	SGD + GAM	SAM	SAM + GAM
ResNet18	Basic	95.32$_{\pm 0.13}$	**96.17**$_{\pm 0.21}$	96.10$_{\pm 0.20}$	**96.75**$_{\pm 0.18}$	78.32$_{\pm 0.32}$	**79.53**$_{\pm 0.30}$	79.27$_{\pm 0.16}$	**80.45**$_{\pm 0.25}$
ResNet18	Cutout	95.99$_{\pm 0.13}$	**96.46**$_{\pm 0.20}$	96.64$_{\pm 0.13}$	**96.99**$_{\pm 0.23}$	78.73$_{\pm 0.13}$	**79.89**$_{\pm 0.31}$	79.43$_{\pm 0.15}$	**80.80**$_{\pm 0.14}$
ResNet18	RA	96.07$_{\pm 0.07}$	**96.52**$_{\pm 0.09}$	96.64$_{\pm 0.17}$	**97.06**$_{\pm 0.13}$	78.62$_{\pm 0.32}$	**79.82**$_{\pm 0.24}$	79.71$_{\pm 0.15}$	**80.97**$_{\pm 0.29}$
ResNet18	AA	96.13$_{\pm 0.05}$	**96.71**$_{\pm 0.07}$	96.75$_{\pm 0.08}$	**97.17**$_{\pm 0.08}$	78.88$_{\pm 0.15}$	**80.56**$_{\pm 0.21}$	80.58$_{\pm 0.25}$	**81.59**$_{\pm 0.24}$
ResNet101	Basic	96.35$_{\pm 0.08}$	**96.98**$_{\pm 0.11}$	96.82$_{\pm 0.16}$	**97.20**$_{\pm 0.15}$	80.47$_{\pm 0.13}$	**82.21**$_{\pm 0.40}$	82.03$_{\pm 0.12}$	**83.13**$_{\pm 0.07}$
ResNet101	Cutout	96.56$_{\pm 0.18}$	**97.22**$_{\pm 0.05}$	97.07$_{\pm 0.08}$	**97.36**$_{\pm 0.24}$	80.53$_{\pm 0.30}$	**82.36**$_{\pm 0.24}$	81.60$_{\pm 0.35}$	**83.40**$_{\pm 0.13}$
ResNet101	RA	96.68$_{\pm 0.25}$	**97.33**$_{\pm 0.30}$	97.12$_{\pm 0.18}$	**97.40**$_{\pm 0.23}$	80.60$_{\pm 0.28}$	**82.40**$_{\pm 0.31}$	82.19$_{\pm 0.34}$	**83.28**$_{\pm 0.20}$
ResNet101	AA	96.78$_{\pm 0.14}$	**97.39**$_{\pm 0.18}$	97.18$_{\pm 0.11}$	**97.42**$_{\pm 0.1}$	81.83$_{\pm 0.37}$	**83.19**$_{\pm 0.15}$	82.44$_{\pm 0.47}$	**83.94**$_{\pm 0.23}$
WRN28_2	Basic	94.82$_{\pm 0.07}$	**95.69**$_{\pm 0.13}$	95.47$_{\pm 0.08}$	**95.85**$_{\pm 0.08}$	75.45$_{\pm 0.25}$	**77.21**$_{\pm 0.31}$	77.04$_{\pm 0.18}$	**77.69**$_{\pm 0.20}$
WRN28_2	Cutout	95.70$_{\pm 0.20}$	**96.41**$_{\pm 0.18}$	96.22$_{\pm 0.13}$	**96.39**$_{\pm 0.22}$	76.80$_{\pm 0.45}$	**78.58**$_{\pm 0.24}$	78.04$_{\pm 0.43}$	**79.33**$_{\pm 0.12}$
WRN28_2	RA	95.75$_{\pm 0.16}$	**96.35**$_{\pm 0.13}$	96.22$_{\pm 0.08}$	**96.49**$_{\pm 0.20}$	76.73$_{\pm 0.27}$	**78.66**$_{\pm 0.03}$	77.88$_{\pm 0.29}$	**78.96**$_{\pm 0.13}$
WRN28_2	AA	95.44$_{\pm 0.06}$	**95.98**$_{\pm 0.09}$	96.07$_{\pm 0.08}$	**96.44**$_{\pm 0.09}$	77.35$_{\pm 0.02}$	**79.05**$_{\pm 0.10}$	78.64$_{\pm 0.23}$	**79.50**$_{\pm 0.21}$
WRN28_10	Basic	95.73$_{\pm 0.10}$	**96.61**$_{\pm 0.15}$	96.78$_{\pm 0.80}$	**97.29**$_{\pm 0.11}$	81.40$_{\pm 0.13}$	**83.45**$_{\pm 0.09}$	83.41$_{\pm 0.04}$	**84.31**$_{\pm 0.06}$
WRN28_10	Cutout	96.74$_{\pm 0.03}$	**96.97**$_{\pm 0.05}$	97.35$_{\pm 0.16}$	**97.56**$_{\pm 0.12}$	81.53$_{\pm 0.40}$	**83.69**$_{\pm 0.08}$	82.38$_{\pm 0.15}$	**84.43**$_{\pm 0.13}$
WRN28_10	RA	**97.14**$_{\pm 0.04}$	96.83$_{\pm 0.03}$	**97.58**$_{\pm 0.07}$	97.49$_{\pm 0.03}$	81.65$_{\pm 0.18}$	**83.84**$_{\pm 0.09}$	82.79$_{\pm 0.06}$	**84.68**$_{\pm 0.13}$
WRN28_10	AA	96.93$_{\pm 0.12}$	**97.05**$_{\pm 0.04}$	97.48$_{\pm 0.06}$	**97.67**$_{\pm 0.08}$	81.99$_{\pm 0.11}$	**84.02**$_{\pm 0.18}$	83.84$_{\pm 0.30}$	**84.81**$_{\pm 0.21}$
PyramidNet110	Basic	96.19$_{\pm 0.11}$	**97.11**$_{\pm 0.14}$	97.26$_{\pm 0.05}$	**97.51**$_{\pm 0.09}$	82.74$_{\pm 0.12}$	**84.91**$_{\pm 0.09}$	85.01$_{\pm 0.09}$	**85.25**$_{\pm 0.06}$
PyramidNet110	Cutout	96.82$_{\pm 0.09}$	**97.32**$_{\pm 0.21}$	97.49$_{\pm 0.06}$	**97.91**$_{\pm 0.14}$	83.31$_{\pm 0.21}$	**85.20**$_{\pm 0.19}$	84.90$_{\pm 0.03}$	**85.46**$_{\pm 0.10}$
PyramidNet110	RA	97.15$_{\pm 0.21}$	**97.80**$_{\pm 0.22}$	97.60$_{\pm 0.09}$	**98.01**$_{\pm 0.10}$	84.04$_{\pm 0.19}$	**86.47**$_{\pm 0.14}$	85.33$_{\pm 0.27}$	**85.64**$_{\pm 0.20}$
PyramidNet110	AA	97.11$_{\pm 0.01}$	**97.85**$_{\pm 0.02}$	97.61$_{\pm 0.14}$	**97.95**$_{\pm 0.10}$	84.48$_{\pm 0.03}$	**85.92**$_{\pm 0.03}$	85.69$_{\pm 0.17}$	**86.35**$_{\pm 0.18}$

Ongoing researches on Sharpness

- Theoretical analysis on Sharpness is not over.
  - Practical Sharpness-Aware Minimization Cannot Converge All the Way to Optima (NeurIPS2023)
  - Why does sharpness-aware minimization generalize better than SGD? (NeurIPS2023)
  - Sharpness-aware minimization leads to low-rank features (NeurIPS2023)
  - Implicit Regularization of Sharpness-Aware Minimization for Scale-Invariant Problems (NeurIPS2024)
  - Explicit Eigenvalue Regularization Improves Sharpness-Aware Minimization (NeurIPS2024)

Part I (Optimization-based methods) Summary

- Optimization method for a given model parameter is very important for robustness.
- Gradient and Hessian information provides us various characteristics on how model parameter responds to a given dataset.
- Sharpness-aware minimization is a good example of optimizer, which provides robustness on model parameter with easy deployment.

Methods for resolving distribution shift

#2 Weight-averaging methods

Improved optimization would help, but…

- It still takes time to train models with various hyper-parameters.
- Picking the individual model, which performs best on a validation set, also takes time.
- Optimal model parameter for each dataset would differ…

We are living in a world of checkpoints.

Classification

The following classification models are available, with or without pre-trained wei

- AlexNet
- ConvNeXt
- DenseNet
- EfficientNet
- EfficientNetV2
- GoogLeNet
- Inception V3
- MaxVit
- MNASNet
- MobileNet V2
- MobileNet V3
- RegNet
- ResNet
- ResNeXt
- ShuffleNet V2
- SqueezeNet
- SwinTransformer
- VGG
- VisionTransformer
- Wide ResNet

Quantized models

The following architectures prov

- Quantized GoogLeNet
- Quantized InceptionV3
- Quantized MobileNet V2
- Quantized MobileNet V3
- Quantized ResNet
- Quantized ResNeXt
- Quantized ShuffleNet V2

Object Detection

The following object detection m

- Faster R-CNN
- FCOS
- RetinaNet
- SSD
- SSDlite

Instance Segmentation

The following instance segmentation m

- Mask R-CNN

Video Classification

- WARNING

The video module is in Beta stage, and

The following video classification models a

- Video MViT
- Video ResNet
- Video S3D
- Video SwinTransformer

Vision and Language Pre-Trained Models

Computer Vision • 31 methods

Edit

Involves models that adapt pre-training to the field of Vision-and-Language (V-L) learning and improve the performance on downstream tasks like visual question answering and visual captioning.

According to Du et al. (2022), information coming from the different modalities can be encoded in three ways: fusion encoder, dual encoder, and a combination of both.

References:

- A Survey of Vision-Language Pre-Trained Models
- Vision Language models: towards multi-modal deep learning

Methods

Add a Method

Method	Year	Papers
PLIP Leveraging medical Twitter to build a visual–language foundation model for pathology AI	2023	3
BLIP BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation	2022	35
OFA OFA: Unifying Architectures, Tasks, and Modalities Through a Simple Sequence-to-Sequence Learning Framework	2022	18
AltCLIP AltCLIP: Altering the Language Encoder in CLIP for Extended Language Capabilities	2022	2
FashionCLIP Contrastive language and vision learning of general fashion concepts	2022	2
OneR Unifying Vision-Language Representation Space with Single-tower Transformer	2022	1
ALIGN Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision	2021	1722
CLIP Learning Transferable Visual Models From Natural Language Supervision	2021	1367

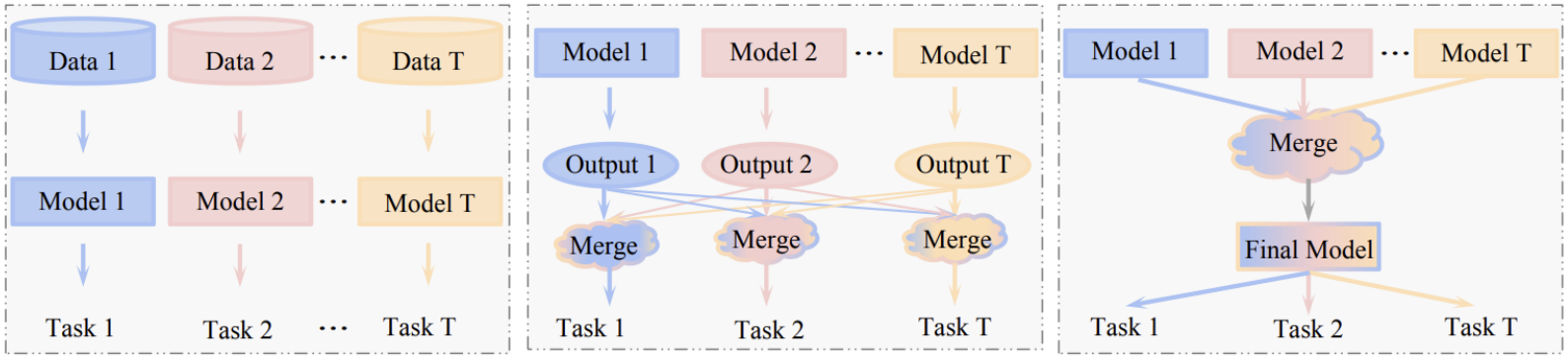
Overview of weight-averaging (or model-merging)

- **Core Idea**
  - Use **multiple sets of model parameters** (weights) obtained from **different datasets**, or **various training strategies** (optimizers/hyperparameters)
  - Combine these weights in a way that boosts final model performance **without incurring additional computational or time costs**.

	Method	Cost
Best on val. set	$f(x, \arg\max_i \text{ValAcc}(\theta_i))$	$\mathcal{O}(1)$
Ensemble	$\frac{1}{k}\sum_{i=1}^{k} f(x, \theta_i)$	$\mathcal{O}(k)$
Uniform soup	$f\left(x, \frac{1}{k}\sum_{i=1}^{k} \theta_i\right)$	$\mathcal{O}(1)$

Overview of weight-averaging (or model-merging)

- **Core Idea**
  - Use **multiple sets of model parameters** (weights) obtained from **different datasets**, or **various training strategies** (optimizers/hyperparameters)



(a) Separate Models (b) Ensemble Learning (c) Model Merging

Overview of weight-averaging (or model-merging)

- **Why Does It Matter?**
  - **Minimal extra training overhead**: We reuse already-existing model weights rather than training a new model from scratch.
  - By merging the strengths of **diverse models**, we often achieve **better generalization** and **robustness** than any single model alone.

	Method	Cost
Best on val. set	$f(x, \arg\max_i \text{ValAcc}(\theta_i))$	$\mathcal{O}(1)$
Ensemble	$\frac{1}{k}\sum_{i=1}^{k} f(x, \theta_i)$	$\mathcal{O}(k)$
Uniform soup	$f\left(x, \frac{1}{k}\sum_{i=1}^{k} \theta_i\right)$	$\mathcal{O}(1)$

SWA (Stochastic Weight averaging)

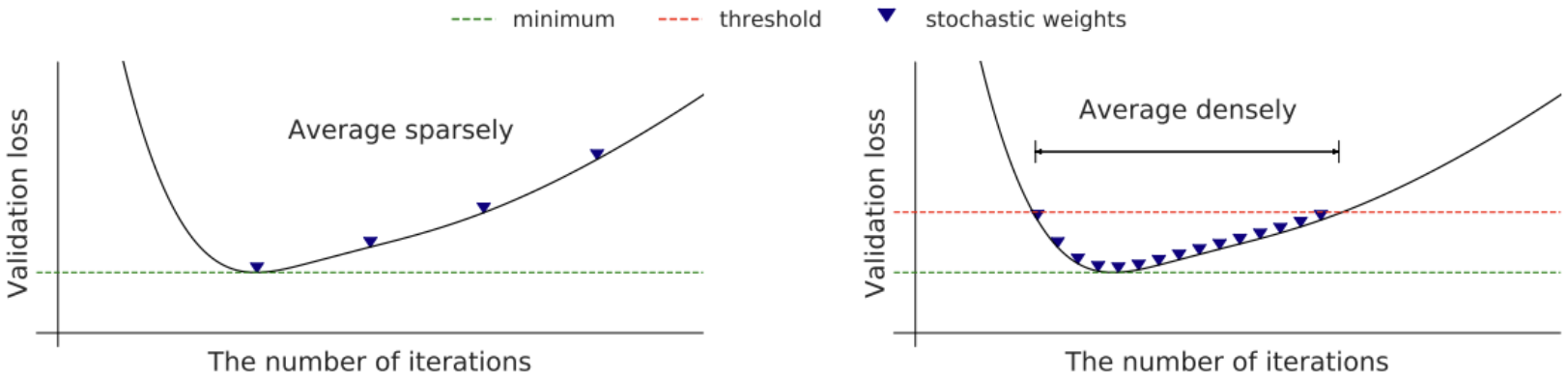
- **Motivation & Insight**
  - **SGD Trajectories** with a cyclical or constant LR tend to *explore* regions of high-performing networks.
  - Averaging these "exploration" checkpoints yields more *robust*, generalizable solutions (lower test error, flatter minima).
- **Key Steps**
  1. **Pretrain** the model (optionally completing or partially completing the usual training schedule).
  2. **Continue training** with cyclical/constant LR to sample diverse weights and **Average** captured weights to form the final SWA model:

$$\theta_{\mathrm{SWA}} = \frac{1}{k}\sum_{i=1}^{k}\theta^{(i)}.$$

SWAD (Variants for SWA for domain generalization)

- **Dense Sampling**: Collects model weights **every iteration** (not just every k epochs) → obtains **many** checkpoints to better capture a wide region of the loss landscape.
- **Overfit-Aware Strategy**: Monitors **validation loss** to identify when the model first stops improving (start iteration) and when it begins overfitting (end iteration). Only weights between them are averaged.
- **Result**:
  - More robust averaging that omits suboptimal, overfitted parameters.
  - Finds an even **flatter minimum** than standard SWA or SAM, leading to **stronger domain generalization**.

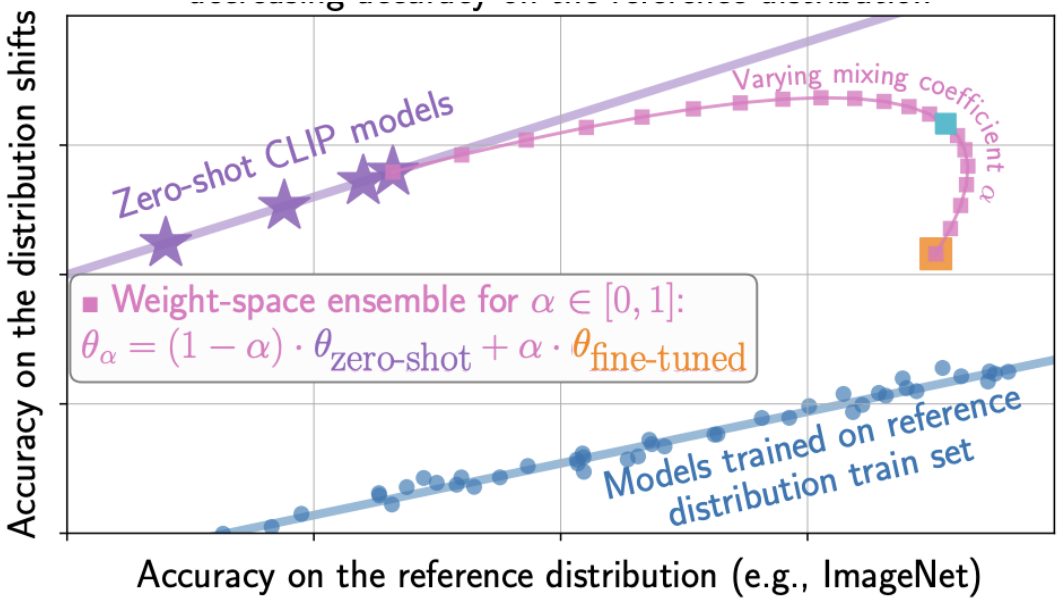
SWA vs SWAD


minimum
threshold
stochastic weights
Validation loss
Average sparsely
The number of iterations
(a) SWA
Validation loss
Average densely
The number of iterations
(b) SWAD (proposed)

Results of SWAD on Domain Generalization

Algorithm	PACS	VLCS	OfficeHome	TerraInc	DomainNet	Avg.
MASF [14]	82.7	-	-	-	-	-
DMG [33]	83.4	-	-	-	43.6	-
MetaReg [15]	83.6	-	-	-	43.6	-
ER [12]	85.3	-	-	-	-	-
pAdaIN [47]	85.4	-	-	-	-	-
EISNet [48]	85.8	-	-	-	-	-
DSON [30]	86.6	-	-	-	-	-
ERM† [29]	85.5	77.5	66.5	46.1	40.9	63.3
ERM (reproduced)	84.2	77.3	67.6	47.8	44.0	64.2
IRM† [20]	83.5	78.6	64.3	47.6	33.9	61.6
GroupDRO† [49]	84.4	76.7	66.0	43.2	33.3	60.7
I-Mixup† [50–52]	84.6	77.4	68.1	47.9	39.2	63.4
MLDG† [13]	84.9	77.2	66.8	47.8	41.2	63.6
CORAL† [31]	86.2	78.8	68.7	47.7	41.5	64.5
MMD† [53]	84.7	77.5	66.4	42.2	23.4	58.8
DANN† [9]	83.7	78.6	65.9	46.7	38.3	62.6
CDANN† [10]	82.6	77.5	65.7	45.8	38.3	62.0
MTL† [54]	84.6	77.2	66.4	45.6	40.6	62.9
SagNet† [32]	86.3	77.8	68.1	48.6	40.3	64.2
ARM† [16]	85.1	77.6	64.8	45.5	35.5	61.7
VREx† [21]	84.9	78.3	66.4	46.4	33.6	61.9
RSC† [55]	85.2	77.1	65.5	46.6	38.9	62.7
Mixstyle [17]	85.2	77.9	60.4	44.0	34.0	60.3
SWAD (ours)	**88.1** (±0.1)	**79.1** (±0.1)	**70.6** (±0.2)	**50.0** (±0.3)	**46.5** (±0.1)	**66.9**

Findings of weight-averaging from various fine-tuned models



- When we average weight between zero-shot models and fine-tuned models, **merged model leads to better accuracy** on the distribution shift.

Model soup : averaging from various fined-tuned weights

- **Motivation**
  - Given a base (pre-trained) model θ_0, we fine-tune it under multiple hyperparameter settings $h_1, \ldots, h_k$ to obtain distinct models $\theta_1, \theta_2, \ldots, \theta_k$.
  - **Conventional Approach**: Pick the single θ_i with the highest validation accuracy and discard others.
  - **Model Soup**: Instead, **average** some or all of these weights to boost performance *without* increasing inference time.

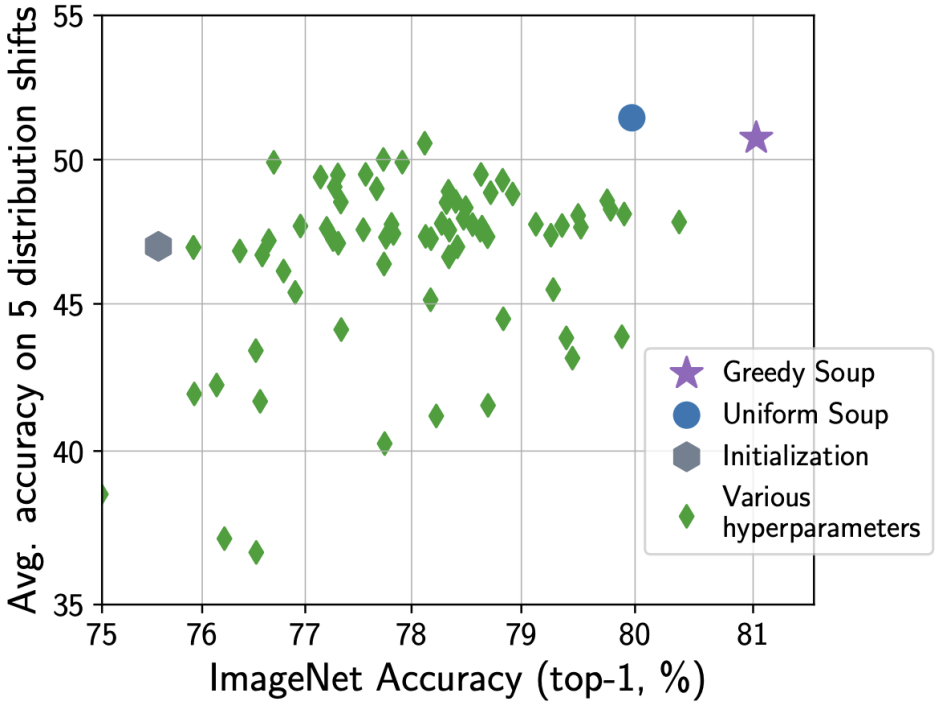
- **Key Idea**
  - The final model has the same inference cost $\mathcal{O}(1)$ as a single model $f(x, \theta_S)$ (unlike ensembles, which cost $\mathcal{O}(k)$).

$$\theta_S = \frac{1}{|S|} \sum_{i \in S} \theta_i, \quad S \subseteq \{1, \ldots, k\}.$$

Model soup : averaging from various fined-tuned weights

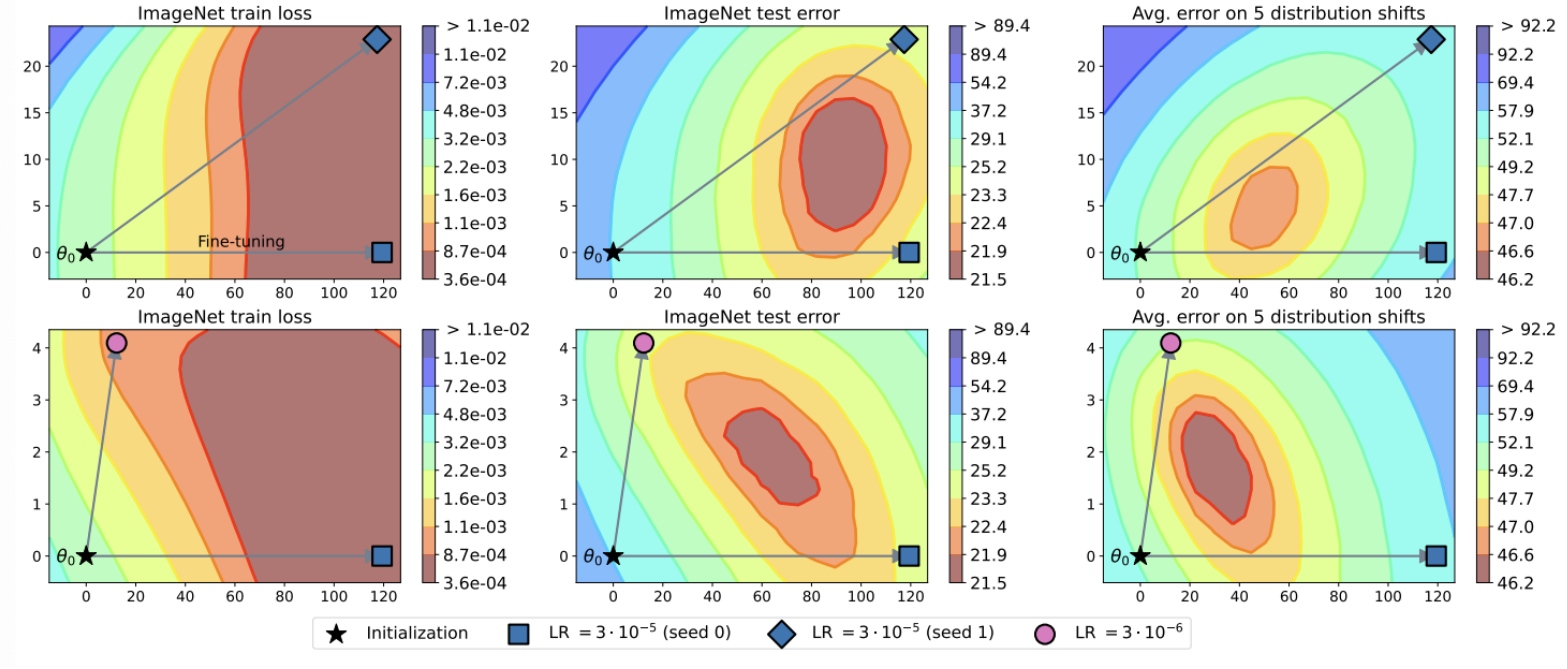
- **Three Recipes for Model Soup**
  1. **Uniform Soup**: Simply average **all** fine-tuned models ($\theta_1, \ldots, \theta_k$).
  2. **Greedy Soup** (Main Method):
     - Sort models by validation accuracy.
     - Iteratively add each model to the soup if it *improves* validation accuracy when averaged with the existing soup.
     - **Result**: Guaranteed not to be worse than the best single model on the validation set.
  3. **Learned Soup**:
     - Optimize interpolation weights via gradient-based methods on a validation set.
     - Potentially more accurate but requires loading all models in memory at once (impractical for very large networks).

Model soup : averaging from various fined-tuned weights



Findings from model soups

- The solution with the highest accuracy is often not a fine-tuned model but rather lies between fine-tuned models.



Model stock

- Gaussian-like weight distributions
  - **Observation**
    - Fine-tuned weights (layer-wise) exhibit geometric patterns suggesting an underlying Gaussian-like behavior in high-dimensional weight space.
    - In high dimensions, vectors sampled from a Gaussian $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$ tend to have nearly identical norms and consistent pairwise angles (concentration of measure).
  - **Key Implication**
    - Fine-tuned weights $\mathbf{w}_1, \ldots, \mathbf{w}_N$ likely lie near a "thin shell" around center .
    - Moving closer to $\boldsymbol{\mu}$ (e.g., through weight merging) can significantly improve performance, especially under distribution shifts or out-of-distribution scenarios.

Model stock : method overview

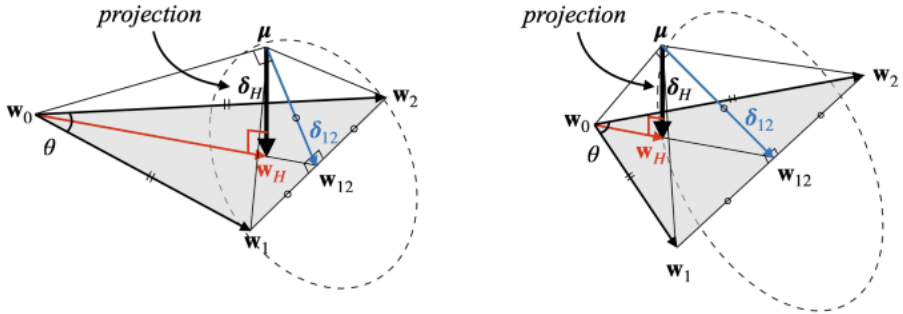
- **Goal**
  - Efficiently approximate the “center” of multiple fine-tuned weights without exhaustive averaging.
  - Leverage a **pre-trained weight** $\mathbf{w}_0$ as a robust **anchor** (observed to preserve out-of-distribution performance).
- **Key Idea**
  1. Identify **geometric relationships** (norms, angles) among $\mathbf{w}_0$ and fine-tuned weights $\mathbf{w}_1, \ldots, \mathbf{w}_N$.
  2. Merge them on a **shared plane** (or space) such that the merged weight $\mathbf{w}_H$ lies closer to the hypothesized center $\boldsymbol{\mu}$ than naive averaging.
  3. The **interpolation ratio** depends **only** on the angle(s) involved, requiring **no extra hyper-parameters** or additional training.

Model stock : method overview

- **Setup**
  - Pre-trained weight $\mathbf{w}_0$, two fine-tuned models $\mathbf{w}_1$ and $\mathbf{w}_2$. All lie in a plane forming a gray triangle ($\triangle \mathbf{w}_0 \mathbf{w}_1 \mathbf{w}_2$).
- **Merged Weight**

$$\mathbf{w}_H = \frac{2\cos\theta}{1+\cos\theta}\mathbf{w}_{12} + \left(1 - \frac{2\cos\theta}{1+\cos\theta}\right)\mathbf{w}_0, \quad \text{where } \mathbf{w}_{12} = \frac{\mathbf{w}_1+\mathbf{w}_2}{2}.$$

  - θ = angle between the two fine-tuned weights. As θ gets smaller (weights are similar), $\mathbf{w}_H$ relies **less** on $\mathbf{w}_0$.

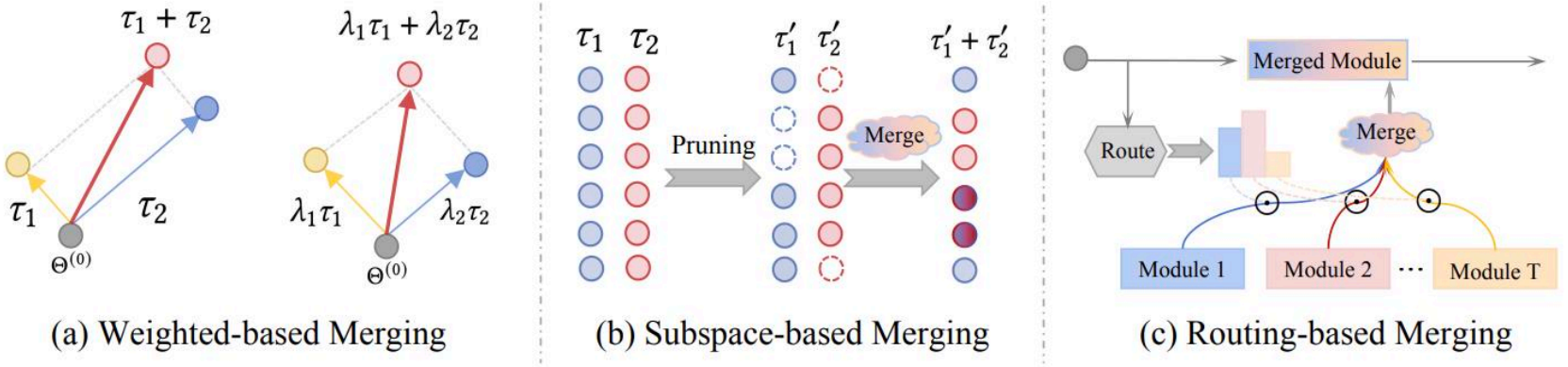


Experimental results of model stock

Table 3: Model Stock on CLIP ViT-B/16. Model Stock shows competitive performance against previous fine-tuning methods on ImageNet and distribution shifts.

Method	ImageNet	Distribution shifts				
		Avg. shifts	IN-V2	IN-R	IN-A	IN-Sketch
Zero-shot	68.3	59.5	62.0	**77.7**	49.9	48.3
Vanilla FT	82.8	57.7	72.9	66.4	43.7	48.0
Vanilla FT*	83.7	57.4	73.5	67.6	40.0	48.6
LP [15]	79.7	48.1	71.5	52.4	27.8	40.5
LP-FT [15]	81.7	60.5	71.6	72.9	49.1	48.4
CAR-FT [23]	83.2	59.4	73.0	71.3	43.7	49.5
FTP [29]	84.2	49.7	74.6	47.2	26.5	50.2
FLYP [7]	82.6	60.5	73.0	71.4	48.1	49.6
Model Stock	84.1	**62.4**	74.8	71.8	**51.2**	**51.8**
Model Stock*	**85.2**	60.1	**75.3**	68.7	45.0	51.3

Beyond simple Weight-based Merging



(a) Weighted-based Merging (b) Subspace-based Merging (c) Routing-based Merging

Part 2 (Weight-averaging methods) Summary

- We could get high performing model based on simple weight merging, without computational complexity.
- How to Merge (or average) model weights could be improved, if we could utilize relationship between each model.

Thank you ☺